

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №6

«Численное решение обыкновенных
дифференциальных уравнений»

по курсу «Вычислительная математика»

Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Малышева Татьяна Алексеевна

Содержание

1	Цель работы и постановка задачи	1
1.1	Задача Коши	1
1.2	Вариант задания	1
1.3	Уравнения, предлагаемые программой	1
2	Описание алгоритма решения задачи	2
2.1	Метод конечных разностей	2
2.2	Общая схема работы программы	2
3	Рабочие формулы используемых методов	4
3.1	Метод Эйлера (метод 1)	4
3.2	Метод Рунге–Кутты 4-го порядка (метод 3)	4
3.3	Метод Адамса, схема предиктор-корректор (метод 4)	4
3.4	Оценка погрешности одношаговых методов: правило Рунге	5
3.5	Оценка погрешности многошагового метода	5
4	Программная реализация	6
4.1	Структура проекта	6
4.2	Ввод исходных данных	6
4.3	Листинг программы	7
5	Результаты выполнения программы	17
5.1	Задача варианта	17
5.2	Правило Рунге	18
5.3	Порядок сходимости	18
5.4	Другие наборы исходных данных	19
5.5	Проверка программы	20
6	Графики точного и приближённого решений	21
7	Скриншоты результатов работы программы	25
8	Вывод	29

1 Цель работы и постановка задачи

Цель работы: решить задачу Коши для обыкновенного дифференциального уравнения первого порядка численными методами, сравнить одношаговые и многошаговые схемы и оценить погрешность полученных решений.

1.1 Задача Коши

Требуется найти функцию $y = y(x)$, удовлетворяющую уравнению первого порядка, разрешённому относительно производной,

$$y' = f(x, y), \quad x \in [x_0, x_n],$$

и начальному условию

$$y(x_0) = y_0.$$

Решить задачу численно — значит для последовательности узлов $x_0 < x_1 < \dots < x_n$ с постоянным шагом $h = x_{i+1} - x_i$ приближённо вычислить значения $y_1, \dots, y_n$ сеточной функции, приближающей точное решение в этих узлах.

Достаточные условия существования и единственности даёт теорема Коши: если $f(x, y)$ непрерывна в замкнутой области $\bar{D} = \{(x, y) : |x - x_0| \leq a, |y - y_0| \leq b\}$, решение существует; если, кроме того, f удовлетворяет условию Липшица по y

$$|f(x, y_1) - f(x, y_2)| \leq k|y_1 - y_2|, \quad k = \text{const} > 0,$$

то решение единственно. Геометрически задача Коши — это поиск интегральной кривой, проходящей через заданную начальную точку $M_0(x_0, y_0)$.

1.2 Вариант задания

Номер варианта — 19. По таблице 1 задания варианту соответствует набор методов **1, 3, 4**:

- одношаговые: **метод Эйлера** (метод 1) и **метод Рунге–Кутты 4-го порядка** (метод 3);
- многошаговый: **метод Адамса** в схеме предиктор-корректор (метод 4).

Точность одношаговых методов оценивается по **правилу Рунге**, точность многошагового — по точному решению: $\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$.

1.3 Уравнения, предлагаемые программой

Задание требует предложить пользователю не менее трёх уравнений. Программа предлагает шесть; для каждого известно аналитическое решение задачи Коши, поэтому погрешность можно измерить в каждом узле.

Таблица 1: Уравнения и точные решения задачи Коши

Идентификатор	$y' = f(x, y)$	Точное решение
linear	y	$y = y_0 e^{x-x_0}$
sum	$x + y$	$y = (y_0 + x_0 + 1)e^{x-x_0} - x - 1$
square	$y - x^2 + 1$	$y = (x + 1)^2 + Ce^x, C = (y_0 - (x_0 + 1)^2)e^{-x_0}$
bernoulli	$y + (1 + x)y^2$	$y = \frac{1}{Ce^{-x} - x}, C = e^{x_0} \left(\frac{1}{y_0} + x_0 \right)$
gauss	$-2xy$	$y = y_0 e^{x_0^2 - x^2}$
stiff	$-5y + 5x^2 + 2x$	$y = x^2 + Ce^{-5x}, C = (y_0 - x_0^2)e^{5x_0}$

Уравнение **bernoulli** совпадает с примером 1 лекции: при $y(1) = -1$ постоянная $C = e \left(\frac{1}{-1} + 1 \right) = 0$, и точное решение принимает вид $y = -\frac{1}{x}$. Это позволяет сверить работу программы с таблицами, приведёнными в лекции, поэтому именно эта задача выбрана в качестве основного демонстрационного набора:

$$y' = y + (1 + x)y^2, \quad y(1) = -1, \quad x \in [1; 1.5], \quad h = 0.1, \quad \varepsilon = 10^{-6}.$$

2 Описание алгоритма решения задачи

2.1 Метод конечных разностей

Все реализованные методы основаны на методе конечных разностей. Область непрерывного изменения аргумента — отрезок $[x_0, x_n]$ — заменяется дискретным множеством узлов $x_i = x_0 + ih$, образующих разностную сетку. Искомая функция непрерывного аргумента заменяется сеточной функцией y_i , а дифференциальное уравнение — разностным уравнением относительно y_i ; входящая в уравнение производная заменяется конечно-разностным отношением. Такая замена называется разностной аппроксимацией на сетке.

2.2 Общая схема работы программы

1. Пользователь выбирает уравнение $y' = f(x, y)$ из предложенного списка.
2. С клавиатуры (либо из файла) вводятся x_0 , $y_0 = y(x_0)$, правая граница x_n , шаг h и точность ε .
3. Входные данные проверяются: $x_n > x_0$, $h > 0$, $h \leq x_n - x_0$, $0 < \varepsilon < 1$, число узлов не превышает предела, правая часть определена в начальной точке.
4. Строится равномерная сетка, содержащая обе границы. Если длина интервала не делится на заданный шаг, фактический шаг немного уменьшается. Вычисляется точное решение $y_{i \text{ точн}}$ в узлах.

5. Задача решается тремя методами варианта. Метод Адамса перед началом счёта получает y_1, y_2, y_3 методом Рунге–Кутта.
6. Составляется общая таблица приближённых значений и погрешностей.
7. Для одношаговых методов выполняется контроль точности по правилу Рунге: задача решается с шагами h и $h/2$, вычисляется R ; пока $R > \varepsilon$, шаг делится пополам. Итоговым становится решение на принятой уточнённой сетке.
8. Для метода Адамса вычисляется $\max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$; пока максимум больше ε , шаг делится пополам и расчёт повторяется.
9. Строятся графики точного и приближённых решений (разными цветами) и график погрешности; результаты выводятся в консоль, в файл и в браузер.

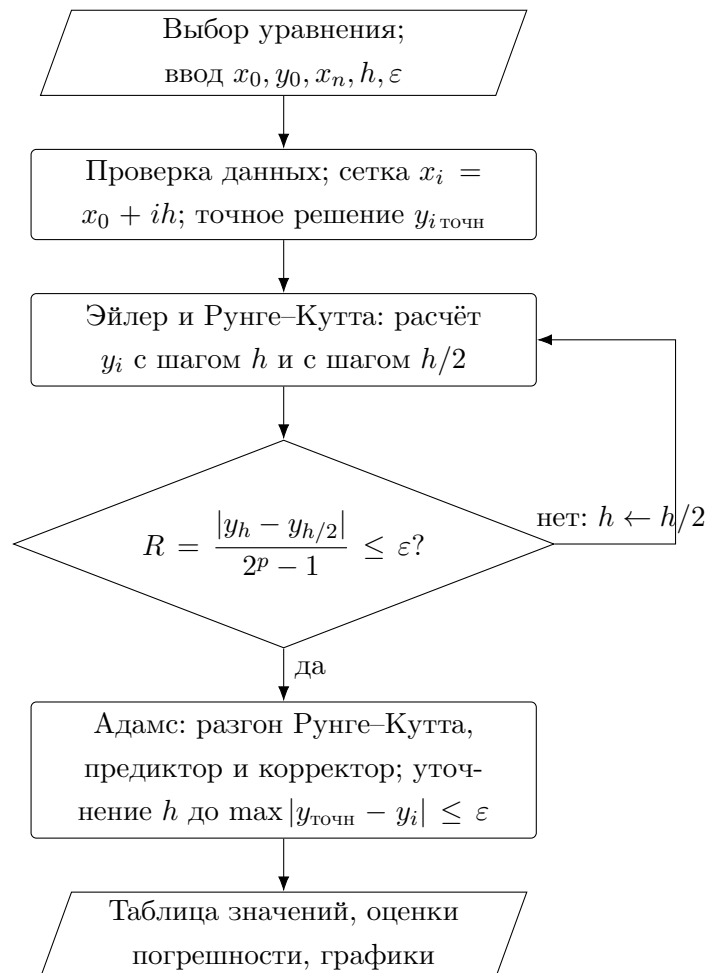


Рис. 1: Блок-схема алгоритма программы

3 Рабочие формулы используемых методов

3.1 Метод Эйлера (метод 1)

Метод основан на разложении решения в ряд Тейлора в окрестности узла x_i с отбрасыванием членов, содержащих производные второго и более высоких порядков:

$$Y(x_i + h) = Y(x_i) + Y'(x_i)h + O(h^2), \quad Y'(x_i) = f(x_i, y_i) \approx \frac{y_{i+1} - y_i}{h}.$$

Отсюда получается расчётная формула

$$y_{i+1} = y_i + hf(x_i, y_i), \quad i = 0, 1, \dots, n-1, \quad (1)$$

$$y_0 = Y_0. \quad (2)$$

Геометрически на каждом шаге приращение решения заменяется приращением ординаты касательной, поэтому после первого шага счёт продолжается уже по другой интегральной кривой. Метод одношаговый, порядок точности $p = 1$, суммарная погрешность $\delta_n = O(h)$.

3.2 Метод Рунге–Кутты 4-го порядка (метод 3)

Идея методов Рунге–Кутты: не использовать частные производные $f(x, y)$, а вычислять саму правую часть в нескольких точках внутри шага. Для наиболее распространённого метода 4-го порядка:

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (3)$$

$$\begin{aligned} k_1 &= hf(x_i, y_i), & k_2 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), & k_4 &= hf(x_i + h, y_i + k_3). \end{aligned} \quad (4)$$

Метод требует четырёхкратного вычисления правой части на каждом шаге, но имеет суммарную погрешность $\delta_n = O(h^4)$, что позволяет считать с существенно большим шагом, чем метод Эйлера. Метод одношаговый: значение y_{i+1} строится только по y_i , поэтому шаг можно менять в любой момент.

3.3 Метод Адамса, схема предиктор-корректор (метод 4)

Многошаговые методы получают интегрированием уравнения на каждом отрезке. Правую часть заменяют интерполяционным многочленом P_{k-1} , построенным по значениям $f_{i-k+1}, \dots, f_i$:

$$y_{i+1} = y_i + \int_{x_i}^{x_{i+1}} P_{k-1}(x) dx.$$

При $k = 4$ и постоянном шаге многочлен Ньютона строится по конечным разностям правой части

$$\Delta f_i = f_i - f_{i-1}, \quad \Delta^2 f_i = f_i - 2f_{i-1} + f_{i-2}, \quad \Delta^3 f_i = f_i - 3f_{i-1} + 3f_{i-2} - f_{i-3},$$

и разностная схема метода Адамса 4-го порядка принимает вид

$$y_{i+1} = y_i + h \left(f_i + \frac{1}{2} \Delta f_i + \frac{5}{12} \Delta^2 f_i + \frac{3}{8} \Delta^3 f_i \right). \quad (5)$$

Раскрыв разности, получаем явную формулу с числовыми коэффициентами. Именно она используется в программе как **этап предиктора**:

$$y_{i+1}^{\text{прогн}} = y_i + \frac{h}{24} (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}). \quad (6)$$

Этап корректора использует неявную формулу, в правую часть которой входит уже уточняемое значение через $f_{i+1} = f(x_{i+1}, y_{i+1})$:

$$y_{i+1}^{(s+1)} = y_i + \frac{h}{24} \left(9f(x_{i+1}, y_{i+1}^{(s)}) + 19f_i - 5f_{i-1} + f_{i-2} \right). \quad (7)$$

Итерации корректора продолжаются, пока $|y_{i+1}^{(s+1)} - y_{i+1}^{(s)}| > \varepsilon$; при выполнении обратного неравенства $|y_{i+1}^{(s+1)} - y_{i+1}^{(s)}| \leq \varepsilon$ выполняется переход к следующему узлу. Начальным приближением $y_{i+1}^{(0)}$ служит предиктор (6).

Формулы (6) и (7) требуют значений в четырёх предыдущих узлах, поэтому счёт может быть начат только с y_4 . Значения y_1, y_2, y_3 получены методом Рунге–Кутты 4-го порядка, y_0 задано начальным условием. Суммарная погрешность метода $\delta_n = O(h^4)$; при этом на каждом шаге правая часть вычисляется всего один-два раза против четырёх у метода Рунге–Кутты.

Замечание. В лекции формула (17) записана в виде $y_{i+1} = y_i + hf_i + \frac{h^2}{2} \Delta f_i + \frac{5h^3}{12} \Delta^2 f_i + \frac{3h^4}{8} \Delta^3 f_i$. Конечные разности $\Delta^k f_i$ уже безразмерны относительно h , поэтому множителем при всех слагаемых должно быть h в первой степени — как в (5). Именно в такой записи (5) раскрывается ровно в (6), что подтверждено автоматическим тестом.

3.4 Оценка погрешности одношаговых методов: правило Рунге

Для метода порядка p решение вычисляется дважды — с шагом h и с шагом $h/2$, — после чего погрешность оценивается как

$$R = \frac{y_h - y_{h/2}}{2^p - 1}, \quad (8)$$

где y_h и $y_{h/2}$ — решения в одном и том же узле. В программе контроль выполняется на всём интервале: R вычисляется во всех общих узлах двух сеток и берётся максимум по модулю. Если $|R| \leq \varepsilon$, точность считается достигнутой; иначе шаг делится пополам и расчёт повторяется. Для метода Эйлера $p = 1$, для метода Рунге–Кутты $p = 4$.

3.5 Оценка погрешности многошагового метода

Для метода Адамса, как требует задание, используется точное решение:

$$\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|. \quad (9)$$

Такая оценка применима именно потому, что все предлагаемые программой уравнения имеют аналитическое решение. Она же вычисляется и для одношаговых методов — это позволяет сравнить фактическую погрешность с оценкой (8). Если для метода Адамса максимум превышает заданное ε , программа делит шаг пополам и повторяет расчёт.

4 Программная реализация

4.1 Структура проекта

Программа написана на JavaScript для Node.js 20+, сторонних зависимостей нет. Численные методы вынесены в отдельный модуль `src/ode.js`: каждый метод — это самостоятельная функция с одинаковой сигнатурой (f, x_0, y_0, x_n, h) , а список `METHODS` связывает метод с его порядком точности и способом оценки погрешности.

Таблица 2: Состав проекта

Путь	Содержимое
<code>src/ode.js</code>	Уравнения, проверка данных, все методы, оценка погрешности
<code>src/io.js</code>	Чтение задачи из файла и форматирование таблиц
<code>src/index.js</code>	Консольное меню, ввод с клавиатуры, параметры запуска
<code>src/plotting-core.js</code>	Построение SVG-графиков решений и погрешности
<code>src/gui-server.js</code>	Локальный сервер веб-интерфейса
<code>src/report.js</code>	Пересчёт результатов, текстовых файлов и графиков
<code>src/ode.test.js</code>	32 автоматических теста
<code>web/</code>	Интерфейс браузера
<code>examples/</code>	5 корректных наборов, текстовый набор, 4 ошибочных
<code>output/, docs/</code>	Результаты запусков, графики, снимки экрана

4.2 Ввод исходных данных

Данные задаются тремя способами:

- с клавиатуры — интерактивное меню выбирает уравнение и запрашивает $x_0, y_0, x_n, h, \varepsilon$ (десятичная запятая допускается);
- из файла — JSON либо текст вида «имя = значение»;
- параметрами командной строки `--equation, --x0, --y0, --xn, --h, --eps`.

Тот же расчётный модуль используется веб-интерфейсом, поэтому консоль и браузер дают одинаковые числа.

4.3 Листинг программы

Ниже приведён расчётный модуль `src/ode.js` целиком: описание уравнений с точными решениями, проверка входных данных, метод Эйлера, метод Рунге–Кутты 4-го порядка, метод Адамса в схеме предиктор-корректор, правило Рунге и оценка по точному решению.

```
1 import { parseNumber } from "./utils.js";
2
3 // Каждое уравнение задаётся правой частью f(x, y) и точным решением задачи Коши.
4 // exact(x0, y0) возвращает функцию y(x) либо null, если условие недопустимо.
5 export const EQUATIONS = [
6   {
7     id: "linear",
8     label: "y' = y",
9     solutionLabel: "y = .y·e^(-xx)",
10    f: (x, y) => y,
11    exact: (x0, y0) => x => y0 * Math.exp(x - x0),
12  },
13  {
14    id: "sum",
15    label: "y' = x + y",
16    solutionLabel: "y = (.y+.x+1)·e^(-xx) - x - 1",
17    f: (x, y) => x + y,
18    exact: (x0, y0) => x => (y0 + x0 + 1) * Math.exp(x - x0) - x - 1,
19  },
20  {
21    id: "square",
22    label: "y' = y - x2 + 1",
23    solutionLabel: "y = (x+1)2 + C·e^x, C = (-y(,x+1)2)·e^(,x)",
24    f: (x, y) => y - x * x + 1,
25    exact: (x0, y0) => {
26      const c = (y0 - (x0 + 1) ** 2) * Math.exp(-x0);
27      return x => (x + 1) ** 2 + c * Math.exp(x);
28    },
29  },
30  {
31    id: "bernoulli",
32    label: "y' = y + (1+x)·y2",
33    solutionLabel: "y = 1/(C·e^(x) - x), C = e^(,x)·(1/,y + ,x)",
34    note: "Пример 1 из лекции: y(1) = -1 на [1; 1.5] даёт y = -1/x.",
35    f: (x, y) => y + (1 + x) * y * y,
36    exact: (x0, y0) => {
37      if (y0 === 0) return x => 0;
38      const c = Math.exp(x0) * (1 / y0 + x0);
39      return x => {
40        const denominator = c * Math.exp(-x) - x;
41        return denominator === 0 ? Number.NaN : 1 / denominator;
42      };
43    },
44    singularity: (x0, y0, xn) => {
```

```

45     if (y0 === 0) return null;
46     const c = Math.exp(x0) * (1 / y0 + x0);
47     const denominator = x => c * Math.exp(-x) - x;
48     const points = [x0, xn];
49     if (c < 0) {
50         const critical = Math.log(-c);
51         if (critical > x0 && critical < xn) points.push(critical);
52     }
53     points.sort((a, b) => a - b);
54     for (let i = 0; i < points.length; i++) {
55         const value = denominator(points[i]);
56         if (Math.abs(value) <= 1e-12 * Math.max(1, Math.abs(points[i]))) return points[i];
57     }
58     if (i === 0) continue;
59     let left = points[i - 1], right = points[i];
60     let leftValue = denominator(left), rightValue = value;
61     if (Math.sign(leftValue) === Math.sign(rightValue)) continue;
62     for (let iteration = 0; iteration < 80; iteration++) {
63         const middle = (left + right) / 2;
64         const middleValue = denominator(middle);
65         if (Math.sign(middleValue) === Math.sign(leftValue)) {
66             left = middle; leftValue = middleValue;
67         } else {
68             right = middle; rightValue = middleValue;
69         }
70     }
71     return (left + right) / 2;
72 }
73 return null;
74 },
75 {
76     id: "gauss",
77     label: "y' = -2·x·y",
78     solutionLabel: "y = .y·e^(.x²-x²)",
79     f: (x, y) => -2 * x * y,
80     exact: (x0, y0) => x => y0 * Math.exp(x0 * x0 - x * x),
81 },
82 {
83     id: "stiff",
84     label: "y' = -5y + 5x² + 2x",
85     solutionLabel: "y = x² + C·e~(5x), C = (.,yx²)·e^(5,x)",
86     note: "Быстро затухающая составляющая: явный метод Эйлера расходится при h > 0.4.",
87     f: (x, y) => -5 * y + 5 * x * x + 2 * x,
88     exact: (x0, y0) => {
89         const c = (y0 - x0 * x0) * Math.exp(5 * x0);
90         return x => x * x + c * Math.exp(-5 * x);
91     },
92 },
93 ];

```

```

94
95 export const MAX_NODES = 200000;
96 export const MAX_REFINEMENTS = 20;
97
98 // Принимает идентификатор либо само уравнение, поэтому validateProblem идемпотентна.
99 export function findEquation(id) {
100   if (id && typeof id === "object") id = id.id;
101   const equation = EQUATIONS.find(item => item.id === id);
102   if (!equation) throw new ErrorНеизвестное(`уравнение «${id}». Доступны: ${EQUATIONS.
     map(e => e.id).join(", ")}.`);
103   return equation;
104 }
105
106 export function validateProblem(input) {
107   if (!input || typeof input !== "object") throw new Error("Требуется описание задачи
     Коши.");
108   const equation = findEquation(input.equation ?? input.equationId);
109   const x0 = parseNumber(input.x0, ".x");
110   const y0 = parseNumber(input.y0, ".y");
111   const xn = parseNumber(input.xn, "xn");
112   const requestedH = parseNumber(input.requestedH ?? input.h, "Шаг h");
113   const epsilon = parseNumber(input.epsilon ?? 1e-6, "Точность ε");
114   if (xn <= x0) throw new Error("Правая граница xn должна быть больше .x.");
115   if (requestedH <= 0) throw new Error("Шаг h должен быть положительным.");
116   if (requestedH > xn - x0) throw new Error("Шаг h не должен превышать длину интервала.");
     ;
117   if (epsilon <= 0) throw new Error("Точность ε должна быть положительной.");
118   if (epsilon >= 1) throw new Error("Точность ε должна быть меньше единицы.");
119   const count = gridStepCount(x0, xn, requestedH);
120   if (count + 1 > MAX_NODES) throw new ErrorСлишком(`мелкий шаг: узлов больше ${
     MAX_NODES}`);
121   const h = (xn - x0) / count;
122   gridNodes(x0, xn, h);
123   const singularity = equation.singularity?.(x0, y0, xn);
124   if (Number.isFinite(singularity)) {
125     throw new ErrorТочное(`решение имеет полюс внутри интервала около x = ${singularity.
       toPrecision(8)}. Измените начальные условия или интервал.`);
126   }
127   const problem = { equation, x0, y0, xn, h, requestedH, epsilon };
128   const first = equation.f(x0, y0);
129   if (!Number.isFinite(first)) throw new Error("Правая часть не определена в начальной
     точке.");
130   return problem;
131 }
132
133 function gridStepCount(x0, xn, h) {
134   const ratio = (xn - x0) / h;
135   const nearest = Math.round(ratio);
136   const almostInteger = Math.abs(ratio - nearest) <= 1e-12 * Math.max(1, Math.abs(ratio))
     ;

```

```

137     return Math.max(1, almostInteger ? nearest : Math.ceil(ratio));
138 }
139
140 // Сетка всегда равномерная, содержит обе границы, а её фактический шаг не больше
    заданного.
141 export function gridNodes(x0, xn, h) {
142     const count = gridStepCount(x0, xn, h);
143     const step = (xn - x0) / count;
144     const nodes = Array.from({ length: count + 1 }, (_, i) => i === count ? xn : x0 + i *
        step);
145     for (let i = 1; i < nodes.length; i++) {
146         if (!(nodes[i] > nodes[i - 1])) throw new Error("Шаг неразличим в машинной
            арифметике при заданном .x.");
147     }
148     return nodes;
149 }
150
151 function guard(value, method, x) {
152     if (!Number.isFinite(value)) {
153         throw new Error(`${method}: решение ушло в бесконечность около x = ${x.toPrecision(6)}
            }. Уменьшите шаг или интервал.`);
154     }
155     return value;
156 }
157
158 // ----- Одношаговые методы -----
159
160 // Метод Эйлера:  $y_{i+1} = y_i + h \cdot f(x_i, y_i)$ . Порядок точности  $p = 1$ .
161 export function euler(f, x0, y0, xn, h) {
162     const nodes = gridNodes(x0, xn, h);
163     const values = [y0];
164     for (let i = 1; i < nodes.length; i++) {
165         const step = nodes[i] - nodes[i - 1];
166         values.push(guard(values[i - 1] + step * f(nodes[i - 1], values[i - 1]), "Метод
            Эйлера", nodes[i]));
167     }
168     return { nodes, values };
169 }
170
171 // Усовершенствованный метод Эйлера метод( Эйлера с пересчётом),  $p = 2$ .
172 // Реализован для сравнения; в вариант 19 не входит.
173 export function improvedEuler(f, x0, y0, xn, h) {
174     const nodes = gridNodes(x0, xn, h);
175     const values = [y0];
176     for (let i = 1; i < nodes.length; i++) {
177         const step = nodes[i] - nodes[i - 1];
178         const slope = f(nodes[i - 1], values[i - 1]);
179         const predictor = values[i - 1] + step * slope;
180         const next = values[i - 1] + step / 2 * (slope + f(nodes[i], predictor));
181         values.push(guard(next, "Усовершенствованный метод Эйлера", nodes[i]));

```

```

182     }
183     return { nodes, values };
184 }
185
186 // Метод РунгеКутты-го4- порядка, p = 4.
187 export function rungeKutta4(f, x0, y0, xn, h) {
188     const nodes = gridNodes(x0, xn, h);
189     const values = [y0];
190     for (let i = 1; i < nodes.length; i++) {
191         const step = nodes[i] - nodes[i - 1];
192         const x = nodes[i - 1], y = values[i - 1];
193         const k1 = step * f(x, y);
194         const k2 = step * f(x + step / 2, y + k1 / 2);
195         const k3 = step * f(x + step / 2, y + k2 / 2);
196         const k4 = step * f(x + step, y + k3);
197         values.push(guard(y + (k1 + 2 * k2 + 2 * k3 + k4) / 6, "Метод РунгеКутты-", nodes[i]));
198     }
199     return { nodes, values };
200 }
201
202 // ----- Многошаговый метод -----
203
204 // Метод Адамса го4- порядка в схеме предикторкорректор-.
205 // Предиктор АдамсБашфорт(-):  $y_{i+1} = y_i + h/24 \cdot (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$ .
206 // Корректор АдамсМоултон(-):  $y_{i+1} = y_i + h/24 \cdot (9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$ ,
207 // итерируется до  $|y^{(k+1)} - y^{(k)}| \leq \epsilon$ . Разгон - методом РунгеКутты-.
208 export const ADAMS_MAX_ITERATIONS = 20;
209
210 export function adams(f, x0, y0, xn, h, epsilon = 1e-9) {
211     const nodes = gridNodes(x0, xn, h);
212     if (nodes.length < 5) {
213         throw new Error("Метод Адамса требует не менее четырёх шагов: возьмите h <= (xn-x0)/4.");
214     }
215     const start = rungeKutta4(f, x0, y0, nodes[3], h);
216     const values = start.values.slice(0, 4);
217     const slopes = values.map((y, i) => f(nodes[i], y));
218     const corrections = [0, 0, 0, 0];
219     for (let i = 3; i < nodes.length - 1; i++) {
220         const step = nodes[i + 1] - nodes[i];
221         let y = values[i] + step / 24 * (55 * slopes[i] - 59 * slopes[i - 1] + 37 * slopes[i - 2] - 9 * slopes[i - 3]);
222         guard(y, "Метод Адамса", nodes[i + 1]);
223         let iterations = 0, delta = Infinity;
224         while (iterations < ADAMS_MAX_ITERATIONS && delta > epsilon) {
225             const corrected = values[i] + step / 24
226                 * (9 * f(nodes[i + 1], y) + 19 * slopes[i] - 5 * slopes[i - 1] + slopes[i - 2]);

```

```

227     guard(corrected, "Метод Адамса", nodes[i + 1]);
228     delta = Math.abs(corrected - y);
229     y = corrected;
230     iterations++;
231 }
232 if (delta > epsilon) {
233     throw new ErrorМетод(` Адамса: корректор не сошёлся за ${ADAMS_MAX_ITERATIONS}
итераций при x = ${nodes[i + 1].toPrecision(6)}. Уменьшите шаг h.`);
234 }
235 values.push(y);
236 slopes.push(f(nodes[i + 1], y));
237 corrections.push(iterations);
238 }
239 return { nodes, values, corrections, startedBy: "РунгеКутта- го4- порядка" };
240 }
241
242 // Конечные разности правой части:  $\Delta f_i$ ,  $\Delta^2 f_i$ ,  $\Delta^3 f_i$  - форма (17) из лекции.
243 export function backwardDifferences(slopes, i) {
244     const [f3, f2, f1, f0] = [slopes[i - 3], slopes[i - 2], slopes[i - 1], slopes[i]];
245     return { d1: f0 - f1, d2: f0 - 2 * f1 + f2, d3: f0 - 3 * f1 + 3 * f2 - f3 };
246 }
247
248 // ----- Оценка погрешности -----
249
250 // Правило Рунге:  $R = (y_h - y_{h/2}) / (2^p - 1)$  в общих узлах двух сеток.
251 export function rungeError(coarse, fine, order) {
252     if (fine.nodes.length !== 2 * coarse.nodes.length - 1) {
253         throw new Error("Для правила Рунге требуются вложенные равномерные сетки с
отношением шагов 2.");
254     }
255     const denominator = 2 ** order - 1;
256     const tolerance = Math.max(1e-9 * Math.abs(coarse.nodes.at(-1) - coarse.nodes[0]),
        Number.MIN_VALUE);
257     let max = 0, at = coarse.nodes[0];
258     for (let i = 0; i < coarse.nodes.length; i++) {
259         const j = 2 * i;
260         if (Math.abs(fine.nodes[j] - coarse.nodes[i]) > tolerance) {
261             throw new Error("Узлы сеток h и h/2 не совпадают.");
262         }
263         const value = Math.abs(coarse.values[i] - fine.values[j]) / denominator;
264         if (value > max) { max = value; at = coarse.nodes[i]; }
265     }
266     return { value: max, at };
267 }
268
269 // Подбор шага по правилу Рунге: h делится пополам, пока R > ε на всём интервале.
270 export function refineByRunge(method, order, problem) {
271     const { equation, x0, y0, xn, epsilon } = problem;
272     const steps = [];
273     let h = problem.h;

```

```

274 let coarse;
275 try {
276     coarse = method(equation.f, x0, y0, xn, h);
277 } catch (error) {
278     return { steps, h, converged: false, reason: error.message };
279 }
280 for (let attempt = 0; attempt <= MAX_REFINEMENTS; attempt++) {
281     if (2 * coarse.nodes.length - 1 > MAX_NODES) {
282         return { steps, h, converged: false, solution: coarse, reason: Достигнут` предел ${
283             MAX_NODES} узлов.` };
284     }
285     let fine;
286     try {
287         fine = method(equation.f, x0, y0, xn, h / 2);
288     } catch (error) {
289         return { steps, h, converged: false, solution: coarse, reason: error.message };
290     }
291     const runge = rungeError(coarse, fine, order);
292     const fineH = (xn - x0) / (fine.nodes.length - 1);
293     steps.push({ h: fineH, comparedH: h, nodes: fine.nodes.length, error: runge.value, at
294         : runge.at, ok: runge.value <= epsilon });
295     if (runge.value <= epsilon) return { steps, h: fineH, converged: true, solution: fine
296         , coarse, runge };
297     coarse = fine;
298     h = fineH;
299 }
300 return { steps, h, converged: false, solution: coarse, reason: Точность` не достигнута
301     за ${MAX_REFINEMENTS} делений шага.` };
302 }
303
304 // Погрешность по точному решению:  $\epsilon = \max |y_i \text{ точн} - y_i|$ .
305 export function exactError(nodes, values, exact) {
306     let max = 0, at = nodes[0];
307     for (let i = 0; i < nodes.length; i++) {
308         const reference = exact(nodes[i]);
309         if (!Number.isFinite(reference)) {
310             throw new ErrorТочное(` решение не определено при x = ${nodes[i].toPrecision(8)}.`)
311             ;
312         }
313         const value = Math.abs(reference - values[i]);
314         if (value > max) { max = value; at = nodes[i]; }
315     }
316     return { value: max, at };
317 }
318
319 export function refineAdams(problem, exact) {
320     const { equation, x0, y0, xn, epsilon } = problem;
321     const steps = [];
322     let h = problem.h;
323     let solution = null;

```

```

319 for (let attempt = 0; attempt <= MAX_REFINEMENTS; attempt++) {
320   if (gridStepCount(x0, xn, h) + 1 > MAX_NODES) {
321     return { steps, h, converged: false, solution, reason: Достигнут` предел ${
MAX_NODES} узлов.` };
322   }
323   try {
324     solution = adams(equation.f, x0, y0, xn, h, Math.min(epsilon, 1e-10));
325     const error = exactError(solution.nodes, solution.values, exact);
326     const actualH = (xn - x0) / (solution.nodes.length - 1);
327     steps.push({ h: actualH, nodes: solution.nodes.length, error: error.value, at:
error.at, ok: error.value <= epsilon });
328     if (error.value <= epsilon) return { steps, h: actualH, converged: true, solution,
error };
329     h = actualH / 2;
330   } catch (error) {
331     steps.push({ h, error: Number.POSITIVE_INFINITY, ok: false, reason: error.message
});
332     h /= 2;
333   }
334 }
335 return { steps, h, converged: false, solution, reason: Точность` не достигнута за ${
MAX_REFINEMENTS} делений шага.` };
336 }
337
338 function sampleAtNodes(solution, targetNodes) {
339   const sourceCount = solution.nodes.length - 1;
340   const targetCount = targetNodes.length - 1;
341   const ratio = sourceCount / targetCount;
342   if (!Number.isInteger(ratio)) throw new Error("Итоговая сетка метода не вложена в
общую сетку.");
343   return targetNodes.map((_, i) => solution.values[i * ratio]);
344 }
345
346 // ----- Сводное решение -----
347
348 export const METHODS = [
349   { id: "euler", label: "Метод Эйлера", short: "Эйлер", kind: "one-step", order: 1, run:
euler },
350   { id: "rk4", label: "Метод РунгеКутты- го4- порядка", short: "РунгеКутта-", kind: "one-
step", order: 4, run: rungeKutta4 },
351   { id: "adams", label: "Метод Адамса предикторкорректор(-)", short: "Адамс", kind: "
multistep", order: 4, run: adams },
352 ];
353
354 export const VARIANT19_METHODS = ["euler", "rk4", "adams"];
355
356 export function solve(input) {
357   const problem = validateProblem(input);
358   const { equation, x0, y0, xn, h, epsilon } = problem;
359   const exact = equation.exact(x0, y0);

```



```

360 const nodes = gridNodes(x0, xn, h);
361 const exactValues = nodes.map(exact);
362 const singular = exactValues.some(value => !Number.isFinite(value));
363 const warnings = [];
364 if (Math.abs(problem.requestedH - problem.h) > 1e-12 * Math.max(1, problem.requestedH))
365   {
366     warnings.pushДля(` равномерной сетки шаг уменьшен с ${problem.requestedH} до ${
367       problem.h}.`);
368   }
369
370 const results = METHODS.map(method => {
371   const base = { id: method.id, label: method.label, short: method.short, kind: method.
372     kind, order: method.order };
373   let initial;
374   try {
375     initial = method.id === "adams"
376       ? method.run(equation.f, x0, y0, xn, h, epsilon)
377       : method.run(equation.f, x0, y0, xn, h);
378   } catch (error) {
379     initial = null;
380   }
381   const baseError = initial ? exactError(initial.nodes, initial.values, exact) : null;
382   const refinement = method.kind === "one-step"
383     ? refineByRunge(method.run, method.order, problem)
384     : refineAdams(problem, exact);
385   if (!refinement.solution) {
386     return { ...base, applicable: false, reason: refinement.reason ?? "Не удалось
387       построить решение." };
388   }
389   const solution = refinement.solution;
390   const error = exactError(solution.nodes, solution.values, exact);
391   if (!refinement.converged) warnings.push(`${method.label}: ${refinement.reason}`);
392   return { ...base, applicable: true, accuracyMet: refinement.converged,
393     nodes: solution.nodes, values: solution.values, commonValues: sampleAtNodes(
394       solution, nodes),
395     h: (xn - x0) / (solution.nodes.length - 1), error,
396     corrections: solution.corrections, base: initial ? { ...initial, error: baseError }
397     : null,
398     runge: method.kind === "one-step" ? refinement : undefined,
399     exactRefinement: method.kind === "multistep" ? refinement : undefined };
400 });
401
402 const spread = (() => {
403   const applicable = results.filter(r => r.applicable);
404   if (applicable.length < 2) return 0;
405   let maximum = 0;
406   for (let i = 0; i < nodes.length; i++) {
407     let minimum = Infinity, maximumInNode = -Infinity, count = 0;
408     for (const result of applicable) {
409       const value = result.commonValues[i];

```

```

404     if (!Number.isFinite(value)) continue;
405     minimum = Math.min(minimum, value);
406     maximumInNode = Math.max(maximumInNode, value);
407     count++;
408 }
409 if (count >= 2) maximum = Math.max(maximum, maximumInNode - minimum);
410 }
411 return maximum;
412 })();
413 return { problem, equation, exact, nodes, exactValues, singular, results, spread,
414         warnings };

```

5 Результаты выполнения программы

5.1 Задача варианта

Основной набор: $y' = y + (1 + x)y^2$, $y(1) = -1$, $[1; 1,5]$, $h = 0,1$, $\varepsilon = 10^{-6}$. Точное решение $y = -1/x$.

Таблица 3: Итоговые решения в общих узлах исходной сетки

i	x_i	Эйлер	Рунге–Кутта	Адамс	$y_{\text{точн}}$
0	1,0	-1,000000	-1,000000	-1,000000	-1,000000
1	1,1	-0,909090	-0,909091	-0,909091	-0,909091
2	1,2	-0,833333	-0,833334	-0,833333	-0,833333
3	1,3	-0,769230	-0,769231	-0,769230	-0,769231
4	1,4	-0,714285	-0,714286	-0,714285	-0,714286
5	1,5	-0,666666	-0,666667	-0,666666	-0,666667

Таблица 4: Фактическая погрешность и её оценка

Метод	p	Способ оценки	Оценка	$\max y_{\text{точн}} - y_i $
Эйлера	1	правило Рунге	$8,57 \cdot 10^{-7}$ при $h = 6,10 \cdot 10^{-6}$	$8,57 \cdot 10^{-7}$
Рунге–Кутта	4	правило Рунге	$2,34 \cdot 10^{-7}$ при $h = 0,05$	$2,15 \cdot 10^{-7}$
Адамса	4	$\max y_{\text{точн}} - y_i $	$9,80 \cdot 10^{-7}$ при $h = 0,05$	$9,80 \cdot 10^{-7}$

Все три строки таблицы относятся к принятым уточнённым сеткам. Для удобства сравнения итоговые решения показаны в шести общих узлах исходной сетки; максимум погрешности вычислен по всем узлам сетки каждого метода.

Отдельный прямой расчёт на исходном шаге $h = 0,1$ совпадает с таблицами примеров 1 и 3 лекции во всех приведённых там знаках; это совпадение закреплёно автоматическими тестами.

Таблица 5: Сравнение с таблицей примера 1 (метод Эйлера) и примера 3 лекции

x_i	Эйлер, лекция	Эйлер, программа	Рунге–Кутта, лекция	Рунге–Кутта, программа
1,1	-0,900000	-0,900000	-0,909093	-0,909093
1,2	-0,819900	-0,819900	-0,833336	-0,833337
1,3	-0,753998	-0,753998	-0,769234	-0,769234
1,4	-0,698640	-0,698640	-0,714289	-0,714289
1,5	-0,651361	-0,651360	-0,666670	-0,666670

Последний знак местами отличается из-за округления в лекции; программа хранит промежуточные значения в двойной точности.

5.2 Правило Рунге

Для метода Рунге–Кутта заданная точность $\varepsilon = 10^{-6}$ достигается после одной проверки сеток $h = 0,1$ и $h/2 = 0,05$: $R = 2,34 \cdot 10^{-7}$. Методу Эйлера потребовалось 14 сравнений сеток.

Таблица 6: Контроль точности метода Эйлера по правилу Рунге ($p = 1$)

h	Узлов	$R = y_h - y_{h/2} /(2^p - 1)$	$R \leq \varepsilon$
0,05	11	$8,27 \cdot 10^{-3}$	нет
0,025	21	$3,79 \cdot 10^{-3}$	нет
0,0125	41	$1,82 \cdot 10^{-3}$	нет
$6,25 \cdot 10^{-3}$	81	$8,94 \cdot 10^{-4}$	нет
$3,125 \cdot 10^{-3}$	161	$4,43 \cdot 10^{-4}$	нет
$1,5625 \cdot 10^{-3}$	321	$2,20 \cdot 10^{-4}$	нет
$7,8125 \cdot 10^{-4}$	641	$1,10 \cdot 10^{-4}$	нет
$3,9063 \cdot 10^{-4}$	1281	$5,49 \cdot 10^{-5}$	нет
$1,9531 \cdot 10^{-4}$	2561	$2,74 \cdot 10^{-5}$	нет
$9,766 \cdot 10^{-5}$	5121	$1,37 \cdot 10^{-5}$	нет
$4,883 \cdot 10^{-5}$	10241	$6,86 \cdot 10^{-6}$	нет
$2,441 \cdot 10^{-5}$	20481	$3,43 \cdot 10^{-6}$	нет
$1,221 \cdot 10^{-5}$	40961	$1,71 \cdot 10^{-6}$	нет
$6,104 \cdot 10^{-6}$	81921	$8,57 \cdot 10^{-7}$	да

Каждое деление шага уменьшает R примерно вдвое — это прямое подтверждение первого порядка точности метода Эйлера.

Для метода Адамса на исходном шаге получено $\max |y_{\text{точн}} - y_i| = 7,45 \cdot 10^{-6}$. После деления шага пополам при $h = 0,05$ максимум стал равен $9,80 \cdot 10^{-7}$, поэтому требуемая точность достигнута.

5.3 Порядок сходимости

Для той же задачи фактическая погрешность $\max |y_{\text{точн}} - y_i|$ измерена при последовательном дроблении шага.

Таблица 7: Порядок сходимости методов на задаче варианта

h	Эйлер	отношение	Рунге–Кутта	отношение
0,1	$1,5646 \cdot 10^{-2}$	—	$3,7232 \cdot 10^{-6}$	—
0,05	$7,3797 \cdot 10^{-3}$	2,12	$2,1480 \cdot 10^{-7}$	17,33
0,025	$3,5972 \cdot 10^{-3}$	2,05	$1,2883 \cdot 10^{-8}$	16,67
0,0125	$1,7762 \cdot 10^{-3}$	2,03	$7,8814 \cdot 10^{-10}$	16,35

При уменьшении шага вдвое погрешность метода Эйлера уменьшается примерно в $2^1 = 2$ раза, метода Рунге–Кутта — примерно в $2^4 = 16$ раз, что соответствует

заявленным порядкам точности.

5.4 Другие наборы исходных данных

Набор 2: $y' = -2xy$, $y(0) = 1$, $[0; 2]$, $h = 0,25$, $\varepsilon = 10^{-6}$. Точное решение $y = e^{-x^2}$; шаг намеренно взят крупным, чтобы участок перегиба был пройден за несколько узлов.

Таблица 8: Прямой расчёт уравнения $y' = -2xy$ на исходной сетке $h = 0,25$

i	x_i	Эйлер	Рунге–Кутта	Адамс	$y_{\text{точн}}$
0	0,00	1,000000	1,000000	1,000000	1,000000
2	0,50	0,875000	0,778797	0,778797	0,778801
4	1,00	0,410156	0,367935	0,367411	0,367879
6	1,50	0,076904	0,105709	0,105459	0,105399
8	2,00	0,002403	0,018711	0,018527	0,018316

На исходной сетке погрешности равны: Эйлер $9,62 \cdot 10^{-2}$, Рунге–Кутта $4,00 \cdot 10^{-4}$, Адамс $4,69 \cdot 10^{-4}$. Автоматический контроль уменьшает шаги; для Эйлера достижению $\varepsilon = 10^{-6}$ препятствует установленный предел в 200,000 узлов, что программа сообщает предупреждением.

Набор 3: $y' = y - x^2 + 1$, $y(0) = 0,5$, $[0; 2]$, $h = 0,2$ — задан текстовым файлом с десятичной запятой. При прямом расчёте на исходной сетке точное решение $y = (x + 1)^2 - \frac{1}{2}e^x$. В конечной точке $x = 2$: Эйлер 4,865785, Рунге–Кутта 5,305363, Адамс 5,305210, точное 5,305472. Погрешности соответственно $4,40 \cdot 10^{-1}$, $1,09 \cdot 10^{-4}$ и $2,62 \cdot 10^{-4}$.

Набор 4: $y' = -5y + 5x^2 + 2x$, $y(0) = 1$, $[0; 3]$, $h = 0,5$, $\varepsilon = 10^{-4}$. Точное решение $y = x^2 + e^{-5x}$ содержит быстро затухающую составляющую. Условие устойчивости явного метода Эйлера $h < 2/5 = 0,4$ нарушено, и решение осциллирует с нарастающей амплитудой.

Таблица 9: Неустойчивость явного метода Эйлера при $h = 0,5$

i	x_i	Эйлер	Рунге–Кутта	$y_{\text{точн}}$
0	0,0	1,000000	1,000000	1,000000
1	0,5	-1,500000	0,979818	0,332085
2	1,0	3,375000	1,554621	1,006738
3	1,5	-1,562500	2,691018	2,250553
4	2,0	9,468750	4,367353	4,000045
5	2,5	-2,203125	6,569585	6,250004
6	3,0	21,429688	9,288611	9,000000

На исходной сетке корректор Адамса не сходится. Программа уменьшает шаг и

повторяет расчёт; при $h = 0,0625$ методы Рунге–Кутта и Адамса, а при $h \approx 6,10 \cdot 10^{-5}$ метод Эйлера достигают $\varepsilon = 10^{-4}$.

5.5 Проверка программы

Таблица 10: Наборы данных для проверки

Набор	Содержание проверки	Результат
Вариант 19	$y' = y + (1 + x)y^2$, $y(1) = -1$, [1; 1,5], $h = 0,1$	расчёт выполнен
Уравнение $y' = -2xy$	[0; 2], крупный шаг $h = 0,25$	расчёт выполнен
Текстовый файл	$y' = y - x^2 + 1$, десятичная запятая	файл прочитан
Жёсткая задача	$y' = -5y + 5x^2 + 2x$, $h = 0,5$	шаг автоматически уменьшен
Крупный шаг	$h > (x_n - x_0)/4$	метод Адамса запущен после уменьшения h
Некорректный набор	$h = -0,1$; $x_n < x_0$; $\varepsilon = 0$; неизвестное уравнение	выведена ошибка

Автоматические тесты проекта проверяют совпадение методов Эйлера и Рунге–Кутта с таблицами примеров 1 и 3 лекции, экспериментальные порядки сходимости, разгон метода Адамса, правило Рунге, чтение обоих форматов файлов и обработку некорректных данных.

6 Графики точного и приближённого решений

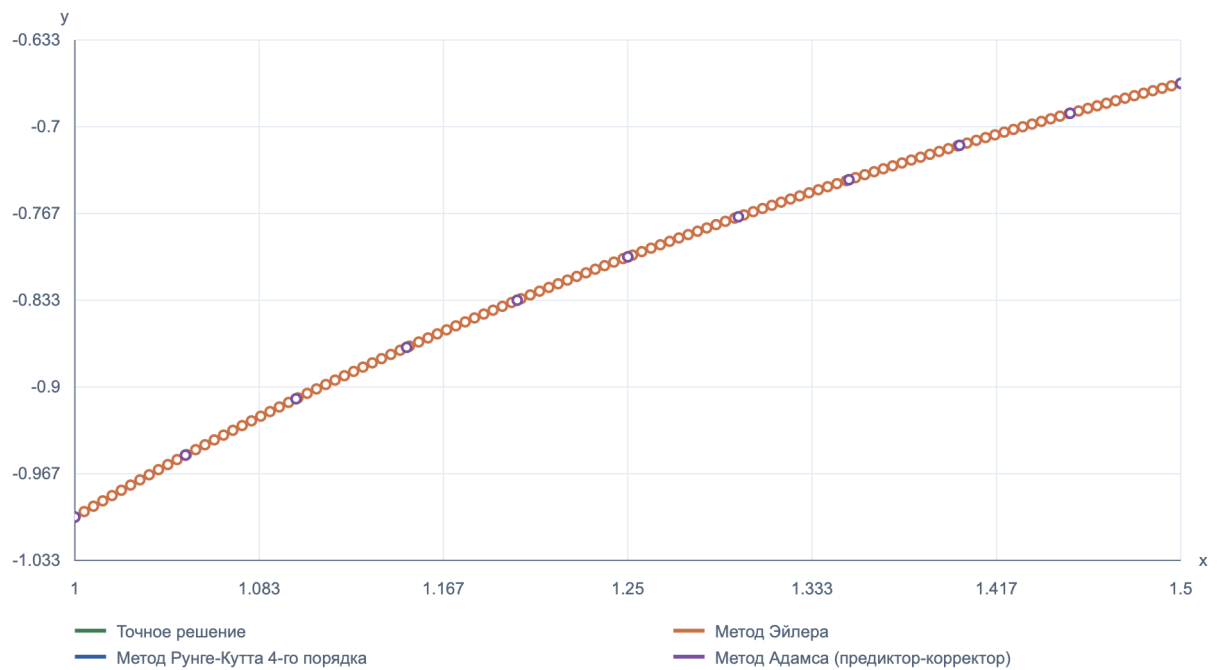


Рис. 2: Задача варианта $y' = y + (1+x)y^2$, $y(1) = -1$: точное решение (зелёная сплошная линия) и принятые приближённые решения (пунктир: оранжевый — Эйлер, синий — Рунге-Кутта, фиолетовый — Адамс)

После контроля точности все три кривые визуально сливаются с точным решением. Методу Эйлера для этого потребовалась значительно более мелкая сетка из-за первого порядка точности.

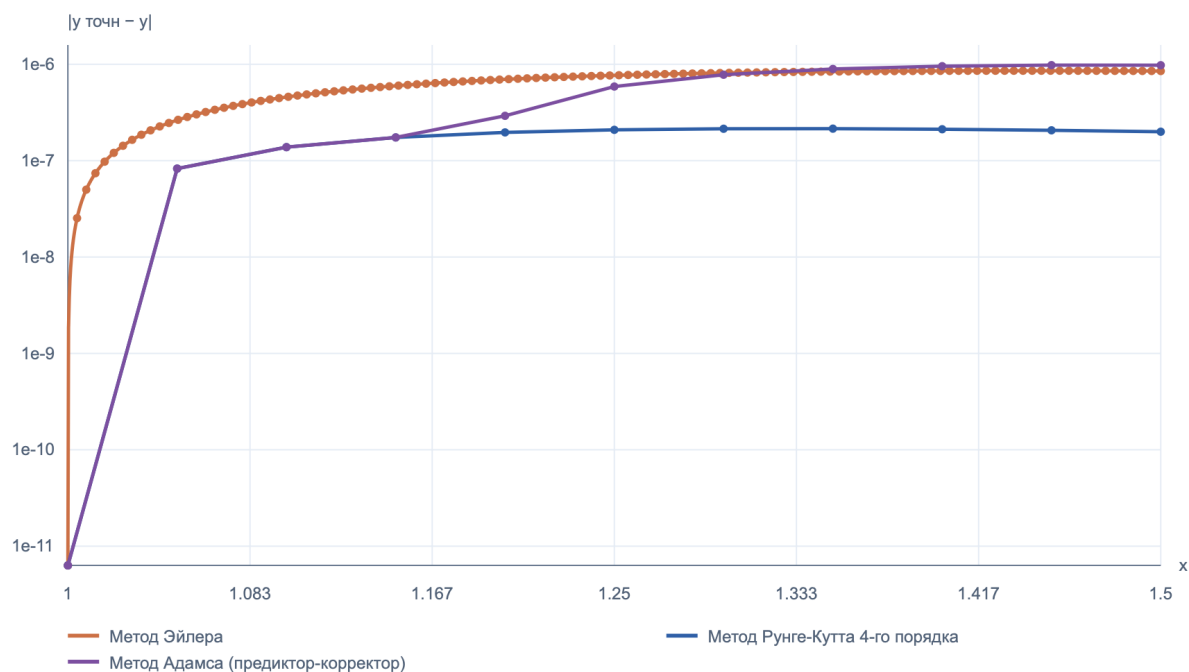


Рис. 3: Погрешность $|y_{\text{точен}} - y_i|$ в узлах, логарифмическая шкала

График погрешности подтверждает, что максимум для каждого принятого решения не превышает 10^{-6} . Разные плотности точек соответствуют индивидуально подобранным шагам методов.

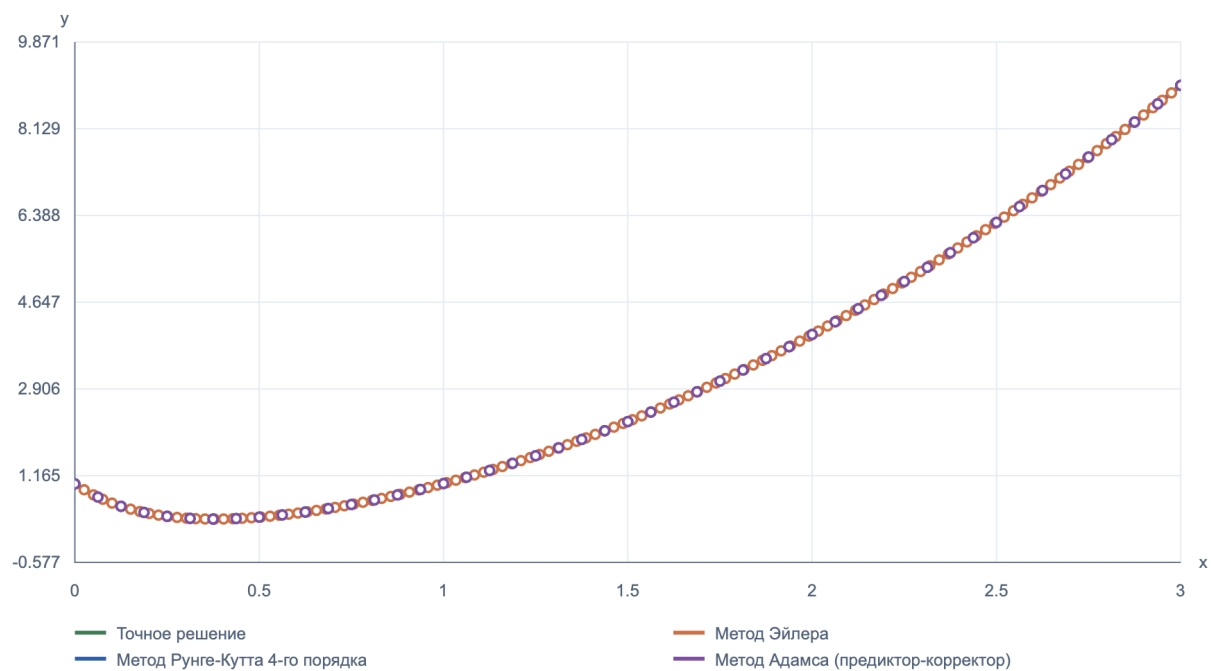


Рис. 4: Жёсткая задача $y' = -5y + 5x^2 + 2x$: решения после автоматического уменьшения исходного шага $h = 0,5$

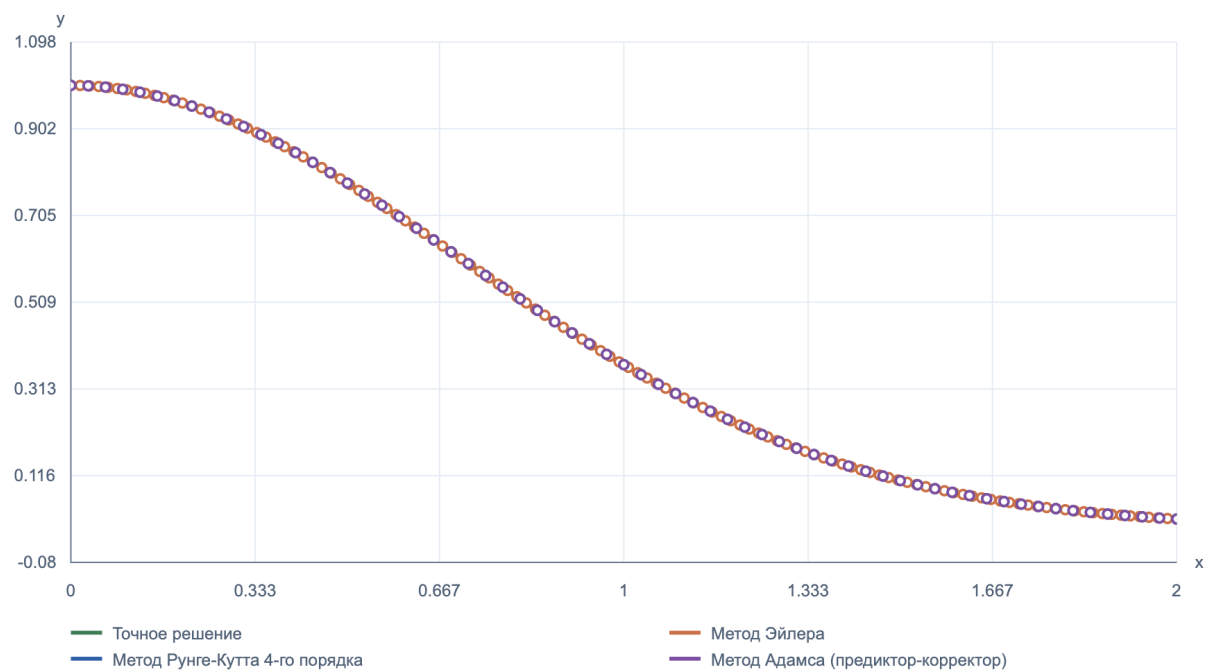


Рис. 5: Уравнение $y' = -2xy$, $y(0) = 1$: решения после контроля точности при исходном шаге $h = 0,25$

7 Скриншоты результатов работы программы

Оформление интерфейса продолжает ЛР №4 и ЛР №5: светлый фон, синие акценты, левая панель ввода и правая область результатов. Ввод x_0 , y_0 , x_n , h , ε и выбор уравнения выполняются в одной форме; таблицы, графики и оценки погрешности обновляются по кнопке «Решить».

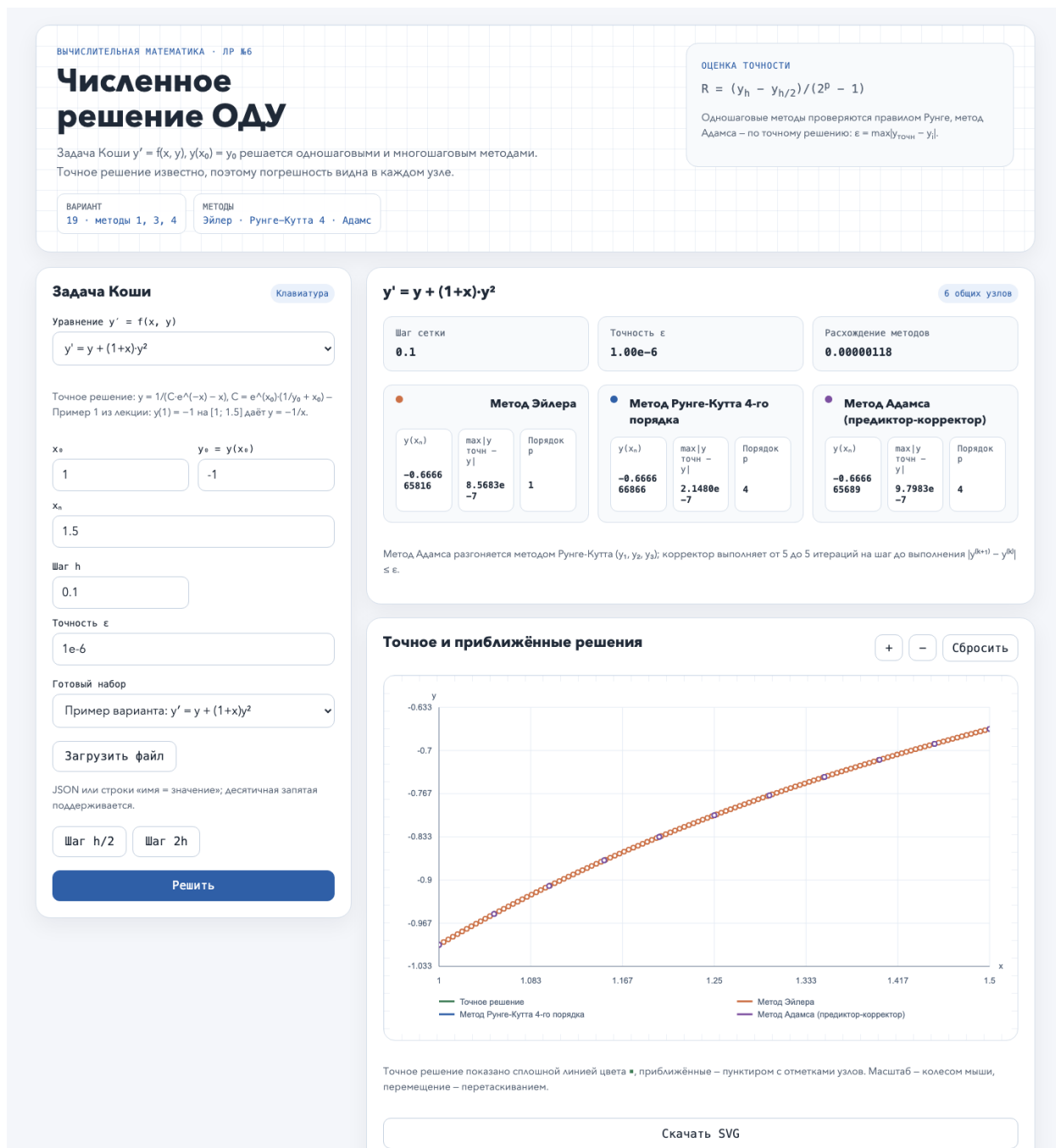


Рис. 6: Набор 1 — задача варианта: параметры, сводка по методам и график точного и приближённых решений

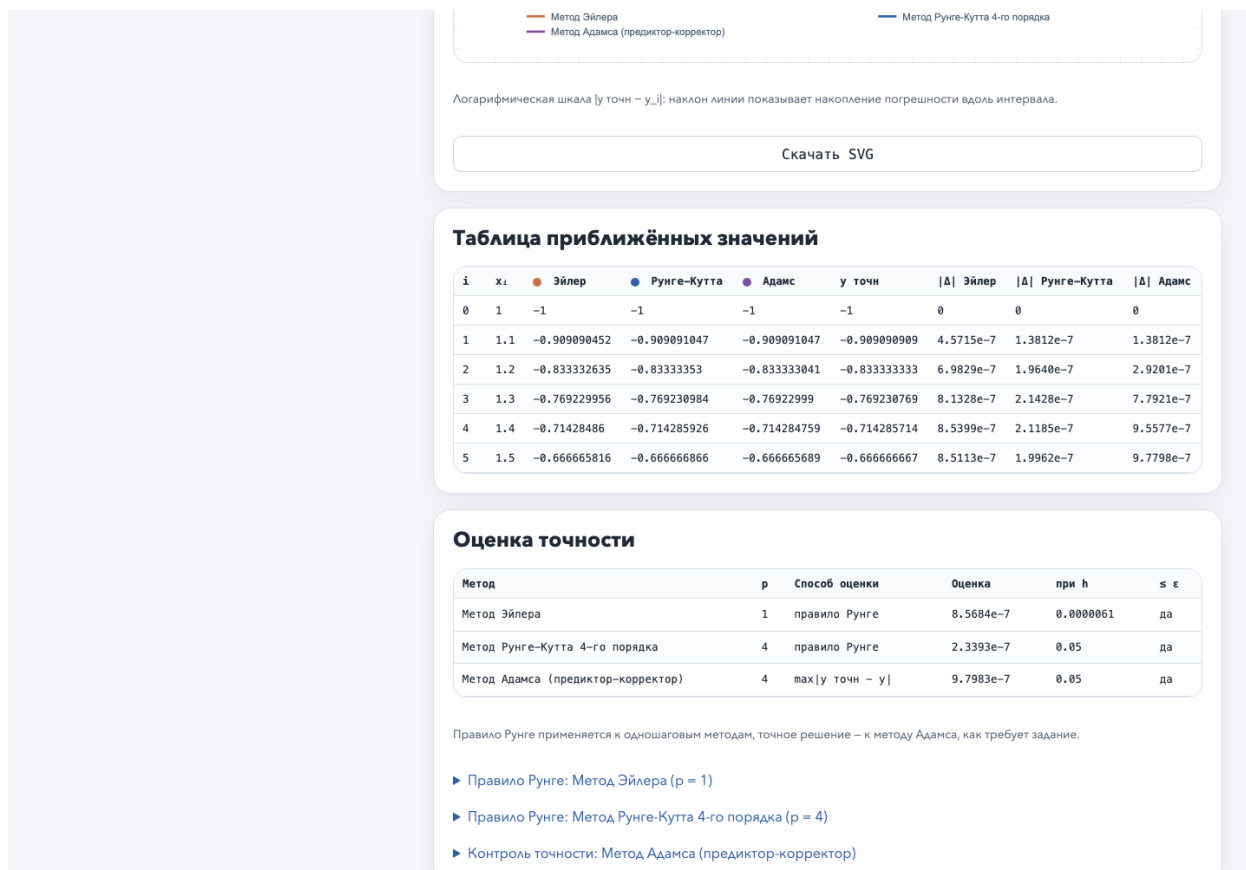


Рис. 7: Тот же запуск: таблица приближённых значений с погрешностью в каждом узле и таблица оценок точности

Численное решение ОДУ

Задача Коши $y' = f(x, y)$, $y(x_0) = y_0$ решается одношаговыми и многошаговыми методами. Точное решение известно, поэтому погрешность видна в каждом узле.

ВАРИАНТ
19 · методы 1, 3, 4

МЕТОДЫ
Эйлер · Рунге-Кутта 4 · Адамс

ОЦЕНКА ТОЧНОСТИ

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению: $\epsilon = \max |y_{\text{точн}} - y_i|$.

Задача Коши

Клавиатура

Уравнение $y' = f(x, y)$

$$y' = y + (1+x)y^2$$

Точное решение: $y = 1/(C \cdot e^{-(1+x)} - x)$, $C = e^{-(x_0)}(1/y_0 + x_0)$ – Пример 1 из лекции: $y(1) = -1$ на $[1; 1.5]$ даёт $y = -1/x$.

x_0 $y_0 = y(x_0)$

x_n

Шаг h

Точность ϵ

Готовый набор

Загрузить файл

JSON или строки «имя = значение»; десятичная запятая поддерживается.

Шаг $h/2$

Шаг $2h$

Решить

Шаг h должен быть положительным.

Данные изменены

Нажмите «Решить», чтобы обновить таблицу и графики.

Рис. 8: Некорректные данные: при $h = -0,1$ расчёт не выполняется, выводится диагностическое сообщение

Численное решение ОДУ

Задача Коши $y' = f(x, y)$, $y(x_0) = y_0$ решается одношаговыми и многошаговыми методами. Точное решение известно, поэтому погрешность видна в каждом узле.

ВАРИАНТ
19 · методы 1, 3, 4

МЕТОДЫ
Эйлер · Рунге-Кутта 4 · Адамс

ОЦЕНКА ТОЧНОСТИ

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению: $\epsilon = \max |y_{\text{точн}} - y_i|$.

Задача Коши

Клавиатура

Уравнение $y' = f(x, y)$

$$y' = y + (1+x)y^2$$

Точное решение: $y = 1/(C \cdot e^{-(1+x)} - x)$, $C = e^{-(1+x_0)}(1/y_0 + x_0)$ – Пример 1 из лекции: $y(1) = -1$ на $[1; 1.5]$ даёт $y = -1/x$.

x_0

1

$y_0 = y(x_0)$

-1

x_n

1.5

Шаг h

0.1

Точность ϵ

1e-6

Готовый набор

Пример варианта: $y' = y + (1+x)y^2$

Загрузить файл

JSON или строки «имя = значение»; десятичная запятая поддерживается.

Шаг $h/2$

Шаг $2h$

Решить

Рис. 9: Адаптивная форма в узком мобильном окне

8 Вывод

Решена задача Коши $y' = y + (1 + x)y^2$ с начальным условием $y(1) = -1$ на отрезке $[1; 1,5]$ с шагом $h = 0,1$ и точностью $\varepsilon = 10^{-6}$; точное решение $y = -1/x$. После автоматического уточнения шага в точке $x = 1,5$ метод Эйлера дал $-0,666666$, метод Рунге–Кутта 4-го порядка — $-0,666667$, метод Адамса — $-0,666666$ при точном значении $-0,666667$. Максимальная фактическая погрешность равна соответственно $8,57 \cdot 10^{-7}$, $2,15 \cdot 10^{-7}$ и $9,80 \cdot 10^{-7}$, то есть все три метода выполнили условие точности.

При $h = 0,05$ те же погрешности равны $7,38 \cdot 10^{-3}$ и $2,15 \cdot 10^{-7}$, то есть в 2,1 и 17,3 раза меньше, чем при $h = 0,1$, что подтверждает первый и четвёртый порядки методов. По правилу Рунге принято решение Рунге–Кутта при $h = 0,05$ (11 узлов), Эйлера — при $h = 6,10 \cdot 10^{-6}$ (81921 узел). Метод Адамса принят при $h = 0,05$; значения y_1 , y_2 , y_3 вычислены методом Рунге–Кутта. Программа диагностирует неверные параметры, полюс точного решения и переполнение, а для шага, не делящего длину интервала, строит равномерную сетку с обеими границами.